Dungeon: Technical Architecture Specification

LLM-Directed Text Adventure with Go Game Engine

March 2026 — v0.2 (design-stage)

Contents

1 Overview **3**
- 1.1 Mechanics Reference . . . 3
- 1.2 Design Principles . . . 3

2 Play Modes **3**
- 2.1 Mode Responsibilities . . . 4

3 System Architecture **4**
- 3.1 Component Map . . . 4
- 3.2 Topology: Embedded vs. Daemon . . . 4
- 3.3 Data Flow: DM Mode . . . 5
- 3.4 Data Flow: Solo Mode . . . 5

4 The Three Engine Interfaces **5**
- 4.1 CommandInterpreter . . . 5
- 4.2 Narrator . . . 6
- 4.3 Renderer . . . 6

5 Game Engine Design **6**
- 5.1 Core Data Model . . . 6
- 5.2 Character . . . 7
- 5.3 Map . . . 7
- 5.4 Combat . . . 7

6 Data Formats **7**
- 6.1 Scenario Format (YAML) . . . 7
- 6.2 Save File Format (JSON) . . . 8

7 MCP Tool Surface (Game Engine) **9**
- 7.1 Session Management . . . 9
- 7.2 Navigation . . . 9
- 7.3 Character and Inventory . . . 9
- 7.4 Combat . . . 10
- 7.5 Search and Interaction . . . 10

8 Lux Display Integration **10**
- 8.1 HUD Layout . . . 10
- 8.2 Update Strategy . . . 11
- 8.3 Lux Tools Used . . . 11

9 mcp-proxy Integration **11**
- 9.1 Problem . . . 11
- 9.2 Solution . . . 11
- 9.3 Daemon Scope Constraint . . . 11

10 SLM Integration **12**

11 Biff Extension Points **12**

12 Implementation Phases **13**

13 Open Questions **13**

14 Test and Verification Architecture **13**
14.1 Philosophy . . . 13
14.2 The Testing Pyramid . . . 14
14.3 Layer 1 — Unit Tests . . . 14
14.3.1 Engine Core . . . 14
14.3.2 Parsers and Formats . . . 15
14.3.3 RulesInterpreter . . . 15
14.3.4 TemplateNarrator . . . 15
14.4 Layer 2 — Integration Tests . . . 15
14.4.1 Engine State Machine Sequences . . . 15
14.4.2 Mock ollama HTTP Server . . . 16
14.4.3 Mock Lux MCP Server . . . 16
14.4.4 Daemon Session Routing . . . 16
14.5 Layer 3 — End-to-End Tests . . . 16
14.6 Layer 4 — Acceptance Tests . . . 17
14.6.1 Game Script Format . . . 17
14.6.2 Bundled Acceptance Scenarios . . . 17
14.7 Test Doubles Reference . . . 18
14.8 Verification Beyond Tests . . . 18
14.8.1 Scenario Validation . . . 18
14.8.2 MCP Schema Contract . . . 18
14.8.3 Race Detection . . . 18
14.8.4 Save Format Forward Compatibility . . . 18
14.9 CI Configuration . . . 18
14.10 What We Do Not Test Automatically . . . 19

1 Overview

Dungeon is a text adventure game for Claude Code. In its current form the entire engine — parser, rules, narrator, state machine — is a single LLM skill prompt. This specification describes the evolved architecture: a deterministic Go game engine that handles all state transitions, with the LLM acting as Dungeon Master (narrator, cartographer, encounter designer) rather than parser.

The central insight is **play mode as a first-class concept**. A play mode is a named combination of three swappable Go interfaces: `CommandInterpreter`, `Narrator`, and `Renderer`. All modes share the same engine, the same save files, and the same scenario format. Switching modes mid-adventure — save in **dm**, resume in **solo** — is explicitly supported.

1.1 Mechanics Reference

The game targets the mechanical vocabulary of *Wizardry I* (grid-based dungeon, turn-based combat, four character classes, dungeon floors with stairs) and the exploratory sensibility of *Zork I* (free-text examination, rich environmental prose, consequence of curiosity). Where the two conflict, Wizardry's determinism wins: rules are authoritative, the engine adjudicates.

1.2 Design Principles

P1. **The engine adjudicates; the LLM narrates.** No game outcome is left to probabilistic language model output. State transitions, combat resolution, and inventory rules are computed deterministically in Go. The LLM enriches the result with prose.

P2. **Interfaces are the seams.** `CommandInterpreter`, `Narrator`, and `Renderer` are the only coupling points between the engine and external systems. Each can be replaced independently.

P3. **Play mode is operational context, not architecture.** The same binary supports DM, solo, and headless play. No code paths are conditional on mode; mode selects implementations.

P4. **Daemon scope stays minimal.** The shared game engine daemon handles game logic and push routing only. No LLM calls, no orchestration, no business logic outside the game model.

P5. **Save files are cross-mode.** A save created in **dm** mode is loadable in **solo** or **headless**. The `play_mode` field in the save is advisory; `--mode` overrides it at resume time.

P6. **Party is designed in, not bolted on.** `GameState.Party` is always a slice (length 1 for single player). Combat, movement, and save tooling accept an optional `character_id`. Multi-player (Biff Phase 5) requires no engine redesign.

2 Play Modes

Three play modes are first-class citizens of the system. Each is a named triple of interface implementations.

Interface	**dm**	**solo**	**headless**
`CommandInterpreter`	`LLMInterpreter`	`SLMInterpreter`	`SLMInterpreter` or `RulesInterpreter`
`Narrator`	`LLMNarrator`	`SLMNarrator`	`TemplateNarrator`
`Renderer`	`LuxRenderer`	`LuxRenderer` or `CLIRenderer`	`CLIRenderer`
Engine deployment	daemon	embedded	embedded
Invoked by	`/dungeon` skill	`dungeon solo`	`dungeon headless`

2.1 Mode Responsibilities

Concern	dm	solo	headless
Room narration	LLM	SLM	Template
Free-text parsing	LLM	SLM	Rules/regex
Scenario generation	LLM interactive	Pre-made only	Pre-made only
Encounter lore	LLM	Pre-made only	Pre-made only
State transitions	Engine	Engine	Engine
Combat resolution	Engine	Engine	Engine
Fog-of-war, save/load	Engine	Engine	Engine
Rendering	Lux	Lux or CLI	CLI

3 System Architecture

3.1 Component Map

Component	Language / Layer	Responsibility
Game engine	Go (L1)	All state transitions; character, map, inventory, combat, save/load. Zero LLM dependencies.
`CommandInterpreter`	Go interface	Maps player free-text to a structured `EngineAction`.
`Narrator`	Go interface	Generates human-readable prose from engine event results.
`Renderer`	Go interface	Presents game state to the player (Lux HUD or ANSI CLI).
MCP server	Go binary (`dungeon serve`)	Exposes game engine as MCP tools; NDJSON over Unix socket in daemon mode; stdio in embedded mode.
SKILL.md	Prompt (L4)	DM role instructions for Claude; narration, scenario generation, lore, examination responses.
mcp-proxy	Go shim (design-stage)	Bridges Claude Code stdio MCP to shared daemon; injects session identity; enables multi-player push.
Lux	Python / ImGui	Native display surface; receives element trees from `LuxRenderer` via MCP tools.
ollama	External service	Serves SLM for `SLMInterpreter` and `SLMNarrator` in **solo**/**headless** modes.

3.2 Topology: Embedded vs. Daemon

The engine runs in one of two deployment topologies depending on play mode. The MCP tool API and all interface implementations are identical in both topologies.

Embedded (solo / headless)

```
player (stdin / Lux)
    |
dungeon solo <-- single process
    |-- Go engine embedded, no socket
    |-- CommandInterpreter (SLM or rules)
    |-- Narrator (SLM or template)
    `-- Renderer (Lux MCP calls / ANSI stdout)
```

Daemon (dm mode / multi-player)

```
Claude Code session (DM)
```

```
    `-- mcp-proxy <10ms, <10MB
            |
            | NDJSON Unix socket
            v
    dungeon serve <-- shared daemon, one per adventure
            |-- GameState (all sessions)
            |-- session identity routing
            `-- tools/list_changed push to DM on player action

Claude Code session (player, future)
    `-- mcp-proxy -----^
```

The daemon holds all game state. It knows which connection is the DM versus a player via session identity injected by mcp-proxy (resolved from the topmost `claude` ancestor PID at connect time). DM-privileged tools are only available to the DM session.

3.3 Data Flow: DM Mode

1. **Button press (navigation/combat).** Player clicks a Lux navigation button. Lux emits an interaction event. `LuxRenderer` calls the engine directly: `move("N")` or `attack(target_id)`. The engine updates state and returns the result. `LuxRenderer` patches the HUD (map fog reveal, HP update) via `lux.update()`. `LLMNarrator` is called with the result to generate flavour text, which is appended to the narration log. *No LLM round-trip occurs for the state transition itself.*

2. **Free-text input.** Player types "I search the walls for a hidden door." `LLMInterpreter` sends the text to Claude with the current room context. Claude returns a structured action: {`action: "search", target: "walls", hint: "door"`}. Engine executes `search(target, hint)`. Result (found/not found, item if any) passed to `LLMNarrator`. Claude generates flavour prose. `LuxRenderer` updates the narration log.

3. **State change push (multi-player, future).** After any player action, the daemon emits `tools/list_changed` to the DM session. Claude Code (DM) notices the notification and calls `get_game_state` to read the delta. DM narrates the result to all player sessions.

3.4 Data Flow: Solo Mode

Solo mode is functionally identical to DM mode with two substitutions: `LLMInterpreter` → `SLMInterpreter` (ollama HTTP, port 11434) and `LLMNarrator` → `SLMNarrator`. The engine runs embedded; there is no daemon or proxy. If ollama is unavailable, `SLMInterpreter` degrades to `RulesInterpreter` automatically.

4 The Three Engine Interfaces

4.1 CommandInterpreter

Maps a player's free-text string to a structured `EngineAction` that the engine can execute deterministically.

```go
type CommandInterpreter interface {
    Interpret(ctx context.Context, input string,
              state GameState) (EngineAction, error)
}
```

Implementation	Used in	Mechanism
`LLMInterpreter`	**dm**	MCP callback to Claude; compact structured prompt; returns JSON action.
`SLMInterpreter`	**solo**, **headless**	ollama HTTP API; phi-3-mini (default); verb/noun classification.
`RulesInterpreter`	**headless** fallback	Regex/keyword matching over known verb–noun pairs; no model required.

4.2 Narrator

Generates human-readable prose from an `EngineEvent`: the result of an action after the engine has resolved it.

```go
type Narrator interface {
    Narrate(ctx context.Context, event EngineEvent,
            state GameState) (string, error)
}
```

Implementation	Used in	Mechanism
`LLMNarrator`	**dm**	Claude generates rich contextual prose; full adventure context provided.
`SLMNarrator`	**solo**	ollama generates short atmospheric text from room seed + event type.
`TemplateNarrator`	**headless**	Go text/template: "You move north. You are in the {room.name}."

4.3 Renderer

Presents the current `GameState` to the player after each turn. Also receives button interaction events in DM/solo modes.

```go
type Renderer interface {
    Render(ctx context.Context, state GameState,
           narration string) error
    Events() <-chan InputEvent // button presses, free-text
}
```

Implementation	Used in	Mechanism
`LuxRenderer`	**dm**, **solo**	Calls Lux MCP tools (`show`, `update`, `recv`) to drive the full HUD.
`CLIRenderer`	**headless**, SSH	ANSI terminal; ASCII map, text HP bar, stdin input loop.

5 Game Engine Design

5.1 Core Data Model

```go
type GameState struct {
    ScenarioID string
    PlayMode string // advisory; --mode overrides on load
    Timestamp string
    Character Character
    Party []Character // len 1 for single-player
    Dungeon DungeonState
    Log []LogEntry
}

type Character struct {
    Name string
    Class CharClass // Fighter | Mage | Thief | Priest
    Level int
    HP, MaxHP int
    MP, MaxMP int
    Stats Stats // STR INT DEX CON WIS CHA
    XP, Gold int
    Inventory []Item
    Equipped Equipment
    Conditions []Condition
}
```

```go
type DungeonState struct {
    CurrentRoom string
    Visited []string
    RoomState map[string]RoomState
}
```

5.2 Character

- **Classes**: Fighter, Mage, Thief, Priest. Class determines HP dice, MP pool, available spells, and XP thresholds.
- **Stats**: STR, INT, DEX, CON, WIS, CHA. Bonuses computed by standard table (8–18 range, D&D-adjacent).
- **Equipment slots**: weapon, armour, ring, amulet.
- **Leveling**: XP thresholds defined per class in scenario YAML. Stat gains on level-up are deterministic.

5.3 Map

- Grid-based dungeon in the Wizardry I style (20×20 floor). Rooms are nodes; corridors are edges.
- **Pre-generated**: the DM creates the room graph before play begins, either by writing scenario YAML directly or via the `/dungeon:create` skill command. Procedural generation is explicitly not used — it produces maps that are inconsistent on backtracking.
- **Fog of war**: unvisited rooms are opaque on the Lux map canvas. Room content is not revealed until the player enters.
- **Room types**: corridor, chamber, stairway (up/down), treasure vault, boss chamber.
- **Door kinds**: open, locked (requires key item), hidden (requires search action), one-way, trapped.

5.4 Combat

Turn-based, deterministic. No probabilities are left to the LLM.

- Initiative roll (DEX modifier + 1d10) determines turn order.
- **Player actions per turn**: Attack, Cast Spell, Use Item, Flee (DEX check), Defend (+AC until next turn).
- **Enemy AI**: patterns encoded in scenario YAML. Supported patterns: `aggressive` (always attack), `cautious` (flee below 30% HP), `support` (buff allies before attacking), `scripted` (fixed action sequence).
- Damage formula: weapon dice + STR modifier vs. target AC.
- Conditions: poisoned, stunned, confused, blessed, hasted. Duration in turns; applied/cleared by spells and items.
- XP is awarded at `end_combat`; loot rolls are seeded from scenario YAML drop tables.

6 Data Formats

6.1 Scenario Format (YAML)

Scenarios are pre-authored YAML files stored in `scenarios/`. The engine parses them at startup; the LLM never sees the raw YAML during play.

scenarios/classic-fantasy-dungeon.yaml (excerpt)

```yaml
id: classic-fantasy-dungeon
title: "The Grimhold Dungeon"
description: "A classic dungeon crawl..."
version: "1.0"
xp_thresholds:
  fighter: [0, 2000, 4000, 8000, 16000]
  mage: [0, 2500, 5000, 10000, 20000]

rooms:
  - id: entrance
    name: "Dungeon Entrance"
    description: "Stone stairs descend into darkness."
    exits:
      south: goblin_lair
    items: []
    enemies: []

  - id: goblin_lair
    name: "Goblin Lair"
    description: "The reek of smoke and old blood."
    exits:
      north: entrance
      east: { room: treasure_vault, door: locked, key: rusty_key }
    enemies:
      - template: goblin
        count: 3
    loot_table:
      - item: rusty_key
        drop_chance: 0.8

enemy_templates:
  goblin:
    hp_dice: "2d6"
    ac: 12
    attack_dice: "1d4"
    xp: 30
    ai: aggressive
```

6.2 Save File Format (JSON)

Save files live at `.dungeon/saves/<slot>.json` (gitignored).

.dungeon/saves/1.json (excerpt)

```json
{
  "version": 1,
  "scenario": "classic-fantasy-dungeon",
  "play_mode": "dm",
  "timestamp": "2026-03-10T18:00:00Z",
  "character": {
    "name": "Aldric", "class": "fighter", "level": 3,
    "hp": 45, "max_hp": 60, "mp": 0, "max_mp": 0,
    "stats": {"str":16,"dex":12,"con":14,"int":9,"wis":10,"cha":11},
    "xp": 1240, "gold": 48,
    "inventory": [],
    "equipped": {"weapon": "rusty_sword"},
    "conditions": []
  },
  "party": [],
  "dungeon": {
    "current_room": "goblin_lair",
    "visited_rooms": ["entrance", "corridor_a", "goblin_lair"],
```

```
    "room_state": {
      "goblin_lair": {"cleared": true, "items_taken": ["rusty_key"]},
      "treasure_vault": {"door_locked": false}
    }
  },
  "adventure_log": []
}
```

`party` is always present (empty for single-player). `play_mode` is advisory; `--mode` overrides on resume.

7 MCP Tool Surface (Game Engine)

The engine exposes all game actions as MCP tools. The interface is identical in embedded and daemon topologies.

7.1 Session Management

Tool	Arguments	Returns
`new_game`	`scenario_id`, `char_name`, `char_class`, `mode?`	Full initial game state
`load_game`	`slot?`, `mode?`	Full game state from save
`save_game`	`slot?`	Save file path
`list_saves`	—	`[{slot, scenario, char, mode, timestamp}]`
`list_scenarios`	—	`[{id, title, description}]`

7.2 Navigation

Tool	Arguments	Returns
`move`	`direction`, `character_id?`	new_room, encounter?, items?, event?
`look`	—	Current room exits, items, enemies
`get_map`	—	Room graph with visited/fog state

7.3 Character and Inventory

Tool	Arguments	Returns
`get_character`	—	Stats, HP/MP, level, XP, gold, conditions
`get_inventory`	—	Item list with weights and equipped slots
`pick_up`	`item_id`	success, weight_remaining
`drop`	`item_id`	success
`equip`	`item_id`	success, stat_delta
`use_item`	`item_id`, `target?`	Effect result
`examine`	`target`	Raw detail (Narrator adds lore)

7.4 Combat

Tool	Arguments	Returns
`get_combat_state`	—	Turn order, HP, conditions
`attack`	`target_id`, `weapon?`, `character_id?`	Roll, damage, target HP, outcome
`cast_spell`	`spell_id`, `target?`, `character_id?`	Effect, MP cost, outcome
`defend`	`character_id?`	Defence bonus until next turn
`use_item_combat`	`item_id`, `target?`, `character_id?`	Effect result
`flee`	`character_id?`	Success (DEX check); penalty on fail
`end_combat`	—	XP gained, loot found

7.5 Search and Interaction

Tool	Arguments	Returns
`search`	`target?`, `hint?`	Found items/secrets, or nothing
`interact`	`object_id`, `action?`	Outcome (lever pulled, door opened. . .)
`rest`	—	HP/MP restored; random encounter check

8 Lux Display Integration

8.1 HUD Layout

The `LuxRenderer` drives a four-panel HUD. Navigation and combat buttons deliver input events directly to the engine — no LLM round-trip is required for button-driven actions.

Lux HUD layout (schematic)

```
+-----------------------------------------------------+
| Menu: [Game] [Character] [Map] |
+----------------------------+----------------------+
| | HP [========--] 45 |
| DUNGEON MAP | MP [----------] 0 |
| (draw canvas, | Fighter Lv3 XP 1k |
| grid rooms, +----------------------+
| fog of war, | EXITS |
| visited/unvisited, | [N] |
| player dot, | [W] [E] |
| locked doors) | [S] |
| | [D stairs] |
+----------------------------+----------------------+
| NARRATION LOG | COMBAT (when active) |
| (markdown, scrolling) | Goblin [=====--] 8hp |
| | [Attack] [Use Item] |
| | [Defend] [Flee] |
| | Round 2 - Your turn |
+----------------------------+----------------------+
```

8.2 Update Strategy

Event	Lux operation
Room transition (new room entered)	`lux.show()` — replace full narration panel, update map fog
HP/MP change during combat	`lux.update()` — patch progress bar elements in-place
Narration text appended	`lux.update()` — append to narration log list
Combat begins/ends	`lux.show()` — activate or deactivate combat panel
Map fog reveal (new room)	`lux.update()` — patch map canvas element
Nav button press (player)	`lux.recv()` event → engine directly

8.3 Lux Tools Used

The `LuxRenderer` uses the following Lux MCP tools:

- `show(scene_id, elements, title?)` — initial HUD render and full scene transitions.
- `update(scene_id, patches)` — incremental HP, MP, fog, and log updates within a scene.
- `recv(timeout?)` — poll for button interaction events (navigation presses, combat actions).

The engine does not use `show_table`, `show_dashboard`, or `show_diagram`; those are data-analysis surfaces.

9 mcp-proxy Integration

Status: design-stage. The mcp-proxy binary does not yet exist. DM mode (Phase 2) should initially use direct stdio MCP and migrate to the proxy pattern when mcp-proxy ships.

9.1 Problem

Claude Code spawns one MCP server process per session via stdio. In DM mode, that process is the game engine daemon. If the DM's Claude session owns the engine process via stdio, a second player connection is architecturally impossible: there is no shared process to connect to, and the DM cannot push events to players.

9.2 Solution

mcp-proxy is a near-zero-cost Go shim (<10 ms startup, <10 MB RSS) that each Claude session spawns via stdio. The shim connects to a pre-running shared daemon over a persistent bidirectional transport (WebSocket or NDJSON Unix socket) and relays MCP messages. At connect time, mcp-proxy resolves the topmost `claude` ancestor PID and injects it as a session identity header.

The daemon uses session identity to:

- Route DM-privileged tools only to the DM session.
- Push `tools/list_changed` to the DM when a player takes an action.
- Route DM narration output to all player sessions.

9.3 Daemon Scope Constraint

The beads project accumulated 70 000 lines of daemon code before being deleted. This daemon must not repeat that mistake. The game daemon handles:

- Game state (authoritative `GameState` for the session).
- MCP tool dispatch (receive call → engine method → return result).
- Push routing (`tools/list_changed`, narration broadcast).

It does not handle: LLM calls, session management, orchestration logic, plugin lifecycle, or anything outside the game model.

10 SLM Integration

The `SLMInterpreter` and `SLMNarrator` call a locally running ollama instance over HTTP.

ollama request (CommandInterpreter)

```
POST http://localhost:11434/api/generate
{
  "model": "phi3",
  "prompt": "...",
  "stream": false,
  "format": "json"
}
```

Property	Value
Default model	phi3 (phi-3-mini, ≈2.3 GB)
Fallback	`RulesInterpreter` if ollama unavailable
Context window	4 096 tokens (sufficient for room context)
Response format	JSON for `SLMInterpreter`; plain text for `SLMNarrator`
Timeout	5 s (`SLMInterpreter`); 10 s (`SLMNarrator`)

`SLMInterpreter` system prompt provides: current room description, available exits, visible items, and a compact action schema. The model returns a JSON action object that the engine consumes directly.

`SLMNarrator` system prompt provides: room seed text from scenario YAML plus the resolved event type and outcome. The model returns 1–3 sentences of atmospheric prose.

11 Biff Extension Points

The engine is party-ready from day one. No redesign is required when Biff multi-player (Phase 5) is added.

- `GameState.Party` is always a `[]Character` slice; single-player sets `len(Party) == 1`.
- `move`, `attack`, `flee`, `defend` all accept an optional `character_id`; omitting it targets the primary character.
- Combat turn order already handles multiple actors.
- Save files carry a `party` array (empty until Biff).

When Biff is added:

- Each Biff participant controls one character in the party.
- `PostToolUse` hooks notify party members of state-change events.
- The DM narrates for the full party.
- Turn tokens are passed via Biff `/write` messages.

12 Implementation Phases

Phase	Deliverable
1 — Engine Core	Go engine (`GameState`, `Character`, `DungeonMap`, `Inventory`, `Combat`); all three interfaces + all implementations except `LLMInterpreter`/`LLMNarrator`; `CLIRenderer` and `LuxRenderer`; `dungeon serve`, `dungeon solo`, `dungeon headless`; scenario YAML schema; `classic-fantasy-dungeon.yaml` migration; save/load: `.dungeon/saves/<slot>.json`.
2 — DM Mode + Lux HUD	`LLMInterpreter` + `LLMNarrator` (MCP callbacks); SKILL.md rewritten for DM role; full Lux HUD (map canvas, HP/MP bars, nav buttons, narration log); `dungeon dm` command; `/dungeon:create` — interactive scenario generation.
3 — Combat System	Turn-based combat: initiative, weapon dice, AC, conditions; enemy AI patterns from scenario YAML; Lux combat panel; XP, leveling, loot rolls; spells (Mage/Priest).
4 — Rich World	Shops, traps (DEX save), cursed items, locked doors; multiple bundled scenarios; DM-generated richer encounters with backstory and puzzles.
5 — Biff Multi-player	Party management (one character per Biff participant); turn tokens via Biff `/write`; shared save state; DM narrates for the full party.

13 Open Questions

Q1. mcp-proxy dependency. DM mode (Phase 2) should use direct stdio MCP initially and migrate to mcp-proxy when that binary ships. Or should Phase 2 wait? The daemon API is identical either way.

Q2. SLM model selection. phi-3-mini (≈2.3 GB), smolLM-1.7B (≈1.0 GB), or mistral-7b-instruct (≈4.1 GB)? Evaluate against command-classification accuracy on a dungeon command corpus.

Q3. Go MCP SDK. `github.com/mark3labs/mcp-go` versus a minimal hand-rolled stdio JSON-RPC layer. The latter is ≈200 lines and avoids a dependency; the former is more complete but less battle-tested.

Q4. Dice notation parser. Use a third-party Go library (e.g. `dicerobot/diceparser`) or hand-roll? The required subset (`NdM+K`) is simple enough to implement in ≈50 lines.

Q5. Permadeath default. Scenario YAML can configure `death: permadeath` or `death: respawn`. What should the built-in default be? Wizardry is permadeath; Zork is not.

14 Test and Verification Architecture

14.1 Philosophy

The single most important property of the test suite is that it tells you immediately and unambiguously whether a change broke something. Three principles follow from this:

P1. Determinism is the invariant. The engine (Section 5) must always produce the same output for the same input. Every test that touches engine logic asserts a deterministic outcome. No probabilistic assertions, no "roughly correct."

P2. Interface seams are injection points. `CommandInterpreter`, `Narrator`, and `Renderer` (Section 4) are the only coupling points to external systems. All tests above unit level use test doubles at these seams. No real LLM, SLM, or Lux instance is ever required to run CI.

P3. Headless mode is the CI workhorse. headless uses `RulesInterpreter` + `TemplateNarrator` + `CLIRenderer` — all zero-dependency implementations. It runs complete game sessions programmatically and is the primary vehicle for integration and end-to-end tests.

14.2 The Testing Pyramid

Layer	Trigger	Target	Coverage
Unit	Every commit	<5 s	Pure engine functions, parsers, `RulesInterpreter`, `TemplateNarrator`
Integration	Every commit	<30 s	Interface compositions, mock ollama, mock Lux, daemon routing
End-to-End	Every PR	<2 min	Real `dungeon` subprocess; MCP wire smoke; save round-trips
Acceptance	Release	<10 min	YAML game scripts through `dungeon headless`; full adventures

All four layers require zero external services. The full unit and integration pass runs in under 30 seconds. If any target is breached, investigate before adding more tests — the usual culprits are sleeping tests, real socket dials, or oversized fixture reads. Slow tests get skipped; skipped tests catch nothing.

14.3 Layer 1 — Unit Tests

Location: `internal/engine/*_test.go`, `internal/interpreter/*_test.go`, `internal/narrator/*_test.go`

Style: Pure functions only. Table-driven. No goroutines, no sockets, no filesystem access outside `testdata/`.

14.3.1 Engine Core

Package	What to test	Key cases
`engine/character`	Stat bonuses, HP/MP calculation, condition apply/expire	All six stat values; boundary cases (8, 18); stacking conditions
`engine/combat`	Initiative order, attack roll, damage, AC reduction, flee DEX check	Hit/miss at boundary (roll == AC); multi-actor initiative
`engine/inventory`	Pick up/drop weight check, equip slot conflict, use-item effect	Full inventory rejection; slot already occupied; consumable self-removes
`engine/map`	Move through open/locked/hidden doors; fog reveal; stairway traversal	Locked without key; hidden pre/post search; one-way return block
`engine/leveling`	XP threshold crossing, stat delta per class	Level 1→2, 9→10; class-specific thresholds
`engine/combat/ai`	Enemy AI pattern execution	`aggressive` always attacks; `cautious` flees at 30% HP; `scripted` follows sequence
`engine/save`	JSON round-trip fidelity	Full `GameState` marshal/unmarshal deep-equal; version field present

Target coverage: `internal/engine` $\geq$ 90% statement coverage; enforced by CI.

14.3.2 Parsers and Formats

Package	What to test
`scenario/parser`	Valid minimal YAML loads cleanly; missing required field → typed error; unknown field → warning, not error; enemy template reference resolves; broken room exit reference → error
`dice/parser`	`1d6`, `2d6+3`, `1d20-1`; boundary rolls (min, max); invalid notation → error
`save/loader`	Load well-formed save; load save with unknown fields (forward compat); wrong `version` → typed migration error

14.3.3 RulesInterpreter

Exhaustive verb/noun coverage. This interpreter calls no model.

RulesInterpreter verb table (representative)

```
"go north" / "n" -> {action: move, direction: N}
"attack goblin" / "a" -> {action: attack, target: "goblin"}
"pick up sword" -> {action: pick_up, item: "sword"}
"search walls" -> {action: search, target: "walls"}
"i" / "inventory" -> {action: get_inventory}
"rest" -> {action: rest}
(unrecognised input) -> {action: unknown, raw: "..."} // not an error
```

14.3.4 TemplateNarrator

Table-driven: event type + room seed → expected substrings.

TemplateNarrator test cases (representative)

```
{event: moved, room_seed: "reeks of smoke"} -> contains seed text
{event: combat_start, enemies: ["goblin x3"]} -> contains enemy count
{event: item_found, item: "rusty_key"} -> contains item name
{event: level_up, new_level: 4} -> contains "level 4"
```

14.4 Layer 2 — Integration Tests

Location: `internal/**/integration_test.go`, `cmd/**/integration_test.go`

Build tag: `-tags integration`

Style: Real implementations wired together; external services replaced with in-process fakes. May use goroutines and in-memory channels. No subprocess spawning; no real network outside `net/http/httptest`.

14.4.1 Engine State Machine Sequences

Multi-step action sequences exercising real engine state. These catch regressions that unit tests miss because they only test individual functions.

Representative integration sequences

```
# Locked door flow
new_game -> move(S) -> move(E) [blocked: locked] ->
pick_up(rusty_key) -> move(E) [now unlocked]
Assert: room progression correct; key consumed

# Full combat cycle
new_game -> move(S) -> {combat begins} ->
attack x N -> end_combat
Assert: HP decrements correctly; XP awarded; loot in inventory

# Level-up crossing
```

```
Accumulate XP across several combats
Assert: level increased; max_HP increased per class table

# Save/load isolation
new_game -> move(S) -> save -> mutate -> load
Assert: loaded state equals pre-mutation state
```

14.4.2 Mock ollama HTTP Server

`net/http/httptest.NewServer` serves canned fixture JSON. Tests assert that:

- `SLMInterpreter` correctly parses a JSON action from a canned response.
- HTTP timeout → automatic fallback to `RulesInterpreter` (not a crash).
- Non-200 response → fallback.
- Malformed JSON → fallback (not panic).

14.4.3 Mock Lux MCP Server

An in-process fake that records all `show()` and `update()` calls and can inject synthetic interaction events (button presses) via a channel. Tests assert that `LuxRenderer`:

- Calls `show()` on scene transitions (room entry, combat start/end); calls `update()` (not `show()`) for incremental HP, MP, and log changes.
- Routes `recv()` button events to the engine as `InputEvent`s without passing through the Narrator.
- Does not call `show()` for every HP tick (regression guard against the performance anti-pattern).

14.4.4 Daemon Session Routing

Start the daemon in-process with two fake client connections. Assign one the DM identity; the other a player identity.

- DM-privileged tools succeed from the DM client; fail from the player client.
- After any player action, `tools/list_changed` is pushed to the DM client only.
- Client disconnection does not crash the daemon.
- Two concurrent game sessions are fully isolated.

14.5 Layer 3 — End-to-End Tests

Location: `e2e/`
Build tag: `-tags e2e`
Style: Black-box subprocess tests via `os/exec`. Requires `dungeon` binary pre-built. Asserts observable outputs: exit codes, JSON responses, save files on disk.

Test	Command	Assert
Help exits clean	`dungeon --help`	Exit 0; usage text present
List scenarios	`dungeon list-scenarios`	JSON array; ≥1 entry
Headless new game	`dungeon headless --scenario minimal`	Exit 0; initial state JSON
Scripted run	`dungeon headless --script minimal-run.yaml`	All steps pass; final state matches
Save round-trip	new_game → save → kill → load	Save file created; state restored
MCP wire smoke	spawn `dungeon serve`; call each tool once	Valid JSON for all tools; no unexpected stderr
Cross-mode save	headless save → `dungeon solo --dry-run`	`GameState` matches expected values

14.6 Layer 4 — Acceptance Tests

Location: `e2e/acceptance/`

Why headless? Acceptance tests must be deterministic. **headless** uses `RulesInterpreter` (no model) and `TemplateNarrator` (no model). The engine outcome is identical for the same scenario and input sequence on every run.

14.6.1 Game Script Format

Acceptance tests are expressed as YAML game scripts. This makes them readable and reviewable independently of Go test infrastructure.

testdata/scripts/complete-run.yaml (excerpt)

```yaml
scenario: minimal
character: {name: TestHero, class: fighter}
steps:
  - input: "go south"
    expect: {current_room: goblin_lair, narration_contains: "goblin"}

  - input: "attack"
    expect: {combat_active: true, enemy_hp_reduced: true}

  - input: "attack"
    repeat_until: {combat_active: false}
    max_iterations: 20
    expect: {xp_gained: true}

  - input: "save"
    expect: {save_file_exists: true}
```

14.6.2 Bundled Acceptance Scenarios

Script	Coverage
`minimal-run.yaml`	Two rooms, one combat, one item, save/load
`combat-full.yaml`	All combat actions: attack, defend, use item, flee, cast spell
`leveling.yaml`	XP crosses level threshold; stat increase verified
`locked-door.yaml`	Find key, unlock door, search reveals hidden passage
`permadeath.yaml`	Character death in permadeath scenario; game-over state asserted

14.7 Test Doubles Reference

All fakes live in `internal/testutil/` and implement the same Go interfaces as real implementations.

External System	Test Double	Used in
LLM (Claude)	`FakeLLMInterpreter` — canned action from fixture	Integration
LLM (Claude)	`FakeLLMNarrator` — fixture string	Integration
SLM (ollama)	`httptest.NewServer` serving fixture JSON	Integration
Lux MCP	`FakeLuxServer` — records calls, injects events	Integration
Daemon socket	In-process fake transport	Integration
CLI stdin/stdout	`bytes.Buffer`	Integration

14.8 Verification Beyond Tests

14.8.1 Scenario Validation

`dungeon validate <file.yaml>` validates a scenario against the engine's schema and resolves all room and template references. Run in CI as a pre-check on any changed `scenarios/*.yaml`. This is the safeguard for DM-authored content — broken references are caught before a player encounters them at runtime.

14.8.2 MCP Schema Contract

The MCP tool definitions are serialised to `testdata/mcp-schema.json` by a generator. CI diffs the generated schema against the committed version:

```
go run ./cmd/dump-mcp-schema > /tmp/schema.json
diff testdata/mcp-schema.json /tmp/schema.json
```

Any accidental API change — renamed tool, added required argument, changed return field — fails the build before any consumer breaks.

14.8.3 Race Detection

`go test -race` is mandatory on CI for all packages that touch the daemon's goroutine model. The daemon handles concurrent MCP connections; a data race here produces silent state corruption, which is the hardest class of bug to diagnose in a running game session.

14.8.4 Save Format Forward Compatibility

When the save format changes, a migration test loads every fixture in `testdata/saves/` (pinned to older format versions) and asserts clean migration to the current format. Old saves must never silently corrupt.

14.9 CI Configuration

.github/workflows/test.yml (target layout)

```
unit-and-integration:
  - go test -race -count=1 ./...
  - go test -race -tags integration -count=1 ./...
  - go test -cover -coverprofile=coverage.out ./internal/engine/...
  - go tool cover -func=coverage.out # fail if engine < 90%

e2e:
  - go build -o dungeon ./cmd/dungeon
  - go test -tags e2e ./e2e/...
```

```
acceptance:
  runs-on: main and release branches only
  - go build -o dungeon ./cmd/dungeon
  - go test -tags acceptance -timeout 10m ./e2e/acceptance/...
```

14.10 What We Do Not Test Automatically

Thing	Reason
LLM narration quality	Non-deterministic; evaluated by human playtest
SLM command accuracy	Requires real model; evaluated by model eval harness, not CI
Lux visual rendering	ImGui pixel output; evaluated by human inspection
mcp-proxy (pre-ship)	Does not exist yet; contract test stubs only
Balance and pacing	Game design; evaluated by playtesting